

Database Design and the E-R Model

Screen clipping taken: 13-06-2026 08:37

we focus on the entity relationship data model (E-R), which provides a means of identifying entities to be represented in the database and how those entities are related.

The Entity-Relationship Model:

The entity-relationship (E-R) data model was developed to facilitate database design by allowing specification of an enterprise schema that represents the overall logical structure of a database.

The E-R model is very useful in mapping the meanings and interactions of real-world enterprises on to a conceptual schema.

The E-R data model employs three basic concepts:

1. entity sets,
2. relationship sets, and
3. attributes,

The E-R model also has an associated diagrammatic representation, the E-R diagram

Entity Sets:

- An entity is a “thing” or “object” in the real world that is distinguishable from all other objects. For example, each person in a university is an entity.
- An entity has a set of properties, and the values for some set of properties may uniquely identify an entity
- a person may have a person id property whose value uniquely identifies that person.
- An entity set is a set of entities of the same type that share the same properties, or attributes.
- The set of all people who are instructors at a given university, for example, can be defined as the entity set instructor.
- Entity sets do not need to be disjoint. For example, it is possible to define the entity set of all people in a university (person). A person entity may be an instructor entity, a student entity, both, or neither.
- An entity is represented by a set of attributes. Attributes are descriptive properties possessed by each member of an entity set.
- Each entity has a value for each of its attributes. For instance, a particular instructor entity may have the value 12121 for ID, the value Wu for name, the value Finance for dept name, and the value 90000 for salary.

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Figure 7.1 Entity sets *instructor* and *student*.

Screen clipping taken: 13-06-2026 10:22

Relationship Sets:

A relationship is an association among several entities. For example, we can define a relationship advisor that associates instructor Katz with student Shankar. This relationship specifies that Katz is an advisor to student Shankar.

A relationship set is a set of relationships of the same type. Formally, it is a mathematical relation on $n \geq 2$ (possibly non distinct) entity sets. If $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of $\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$ where $(e_1, e_2, \dots, e_n)$ is a relationship.

Consider the two entity sets instructor and student in Figure 7.1. We define the relationship set advisor to denote the association between instructors and students.

The association between entity sets is referred to as participation; that is, the entity sets $E_1, E_2, \dots, E_n$ participate in relationship set R .

A relationship in stance in an E-R schema represents an association between the named entities in the real-world enterprise that is being modeled. The function that an entity plays in a relationship is called that entity's role.

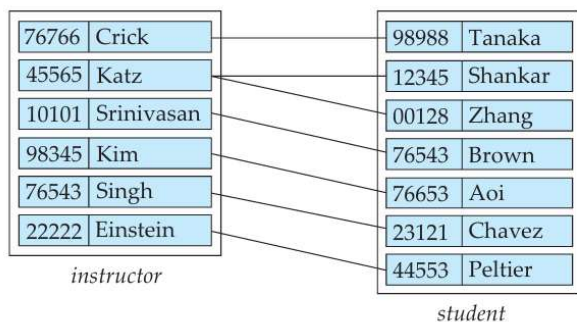


Figure 7.2 Relationship set *advisor*.

Screen clipping taken: 13-06-2026 10:36

Attributes:

- For each attribute, there is a set of permitted values, called the domain, or value set, of that attribute.
- The domain of attribute course id might be the set of all text strings of a certain length.
- the domain of attribute semester might be strings from the set {Fall, Winter, Spring, Summer}.
- Formally, an attribute of an entity set is a function that maps from the entity set into a domain.
- An attribute, as used in the E-R model, can be characterized by the following attribute types.
 - Simple and composite attributes
 - Single-valued and multivalued attributes.
 - Derived attribute.

The attributes have been simple; that is, they have not been divided into subparts.

Composite attributes, on the other hand, can be divided into subparts

Suppose we were to add an address to the student entity-set. The address can be defined as the composite attribute address with the attributes street, city, state, and zip code.

In the composite attribute address, its component attribute street can be further divided into street number, street name, and apartment number.

The attributes in our examples all have a single value for a particular entity. For instance, the student ID attribute for a specific student entity refers to only one student ID. Such attributes are said to be single valued.

There may be instances where an attribute has a set of values for a specific entity.

Suppose we add to the instructor entity set a phone number attribute.

An instructor may have zero, one, or several phone numbers, and different instructors may have different numbers of phones. This type of attribute is said to be multivalued

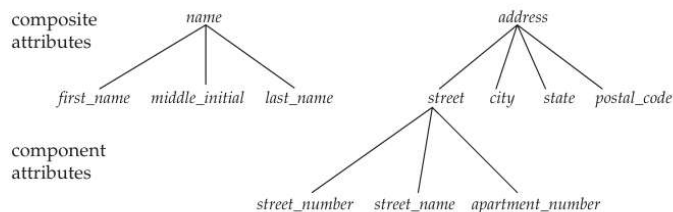


Figure 7.4 Composite attributes instructor *name* and *address*.

Screen clipping taken: 13-06-2026 10:55

Derived attribute. The value for this type of attribute can be derived from the values of other related attributes or entities.

suppose that the instructor entity set has an attribute age that indicates the instructor's age. If the instructor entity set also has an attribute date of birth, we can calculate age from date of birth and the current date. Thus, age is a derived attribute. In this case, date of birth may be referred to as a base attribute, or a stored attribute. The value of a derived attribute is not stored but is computed when required.

An attribute takes a null value when an entity does not have a value for it. The null value may indicate "not applicable"—that is, that the value does not exist for the entity.

Constraints

An E-R enterprise schema may define certain constraints to which the contents of a database must conform

we examine mapping cardinalities and participation constraints.

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set. Mapping cardinalities are most useful in describing binary relationship sets,

For a binary relationship set R between entity sets A and B , the mapping cardinality must be one of the following:

- **One-to-one.** An entity in A is associated with *at most* one entity in B , and an entity in B is associated with *at most* one entity in A . (See Figure 7.5a.)
- **One-to-many.** An entity in A is associated with any number (zero or more) of entities in B . An entity in B , however, can be associated with *at most* one entity in A . (See Figure 7.5b.)

Screen clipping taken: 13-06-2026 16:33

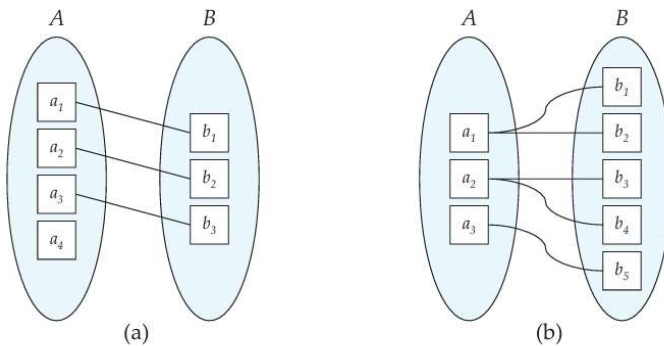


Figure 7.5 Mapping cardinalities. (a) One-to-one. (b) One-to-many.

Screen clipping taken: 13-06-2026 16:34

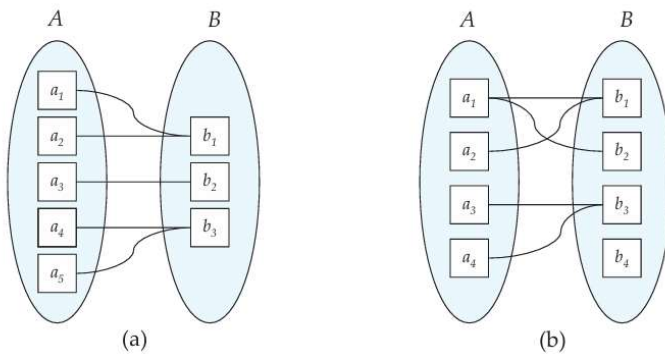


Figure 7.6 Mapping cardinalities. (a) Many-to-one. (b) Many-to-many.

- **Many-to-one.** An entity in A is associated with *at most* one entity in B . An entity in B , however, can be associated with any number (zero or more) of entities in A . (See Figure 7.6a.)
- **Many-to-many.** An entity in A is associated with any number (zero or more) of entities in B , and an entity in B is associated with any number (zero or more) of entities in A . (See Figure 7.6b.)

Screen clipping taken: 13-06-2026 16:48

Participation Constraints The participation of an entity set E in a relationship set R is said to be total if every entity in E participates in at least one relationship in R. If only some entities in E participate in relationships in R, the participation of entity set E in relationship R is said to be partial.

Entity-Relationship Diagrams

Basic Structure :

An E-R diagram consists of the following major components:

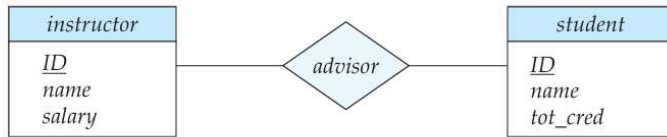


Figure 7.7 E-R diagram corresponding to instructors and students.

Screen clipping taken: 13-06-2026 17:28

- **Rectangles divided into two parts** represent entity sets. The first part, which in this textbook is shaded blue, contains the name of the entity set. The second part contains the names of all the attributes of the entity set.
- **Diamonds** represent relationship sets.
- **Undivided rectangles** represent the attributes of a relationship set. Attributes that are part of the primary key are underlined.
- **Lines** link entity sets to relationship sets.
- **Dashed lines** link attributes of a relationship set to the relationship set.
- **Double lines** indicate total participation of an entity in a relationship set.
- **Double diamonds** represent identifying relationship sets linked to weak entity sets (we discuss identifying relationship sets and weak entity sets later, in Section 7.5.6).

Screen clipping taken: 13-06-2026 17:30

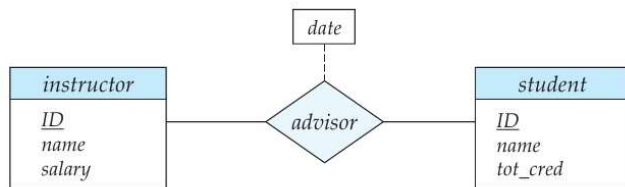


Figure 7.8 E-R diagram with an attribute attached to a relationship set.

Screen clipping taken: 13-06-2026 18:52

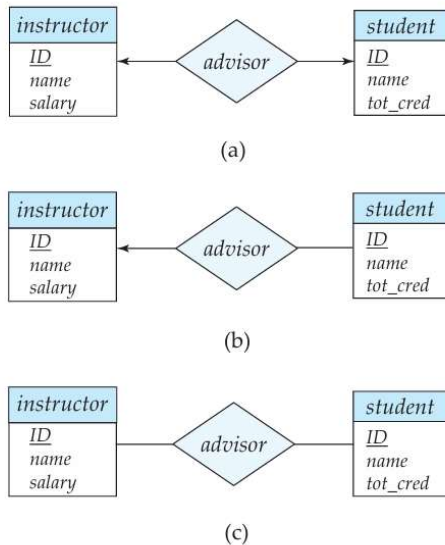


Figure 7.9 Relationships. (a) One-to-one. (b) One-to-many. (c) Many-to-many.

Screen clipping taken: 13-06-2026 18:52

Mapping Cardinality The relationship set *advisor*, between the instructor and student entity sets may be one-to-one, one-to-many, many-to-one, or many-to-many. To distinguish among these types, we draw either a directed line($\rightarrow$) or an undirected line($-$) between the relationship set and the entity set in question, as follows:

- **One-to-one:** We draw a directed line from the relationship set *advisor* to both entity sets *instructor* and *student* (see Figure 7.9a). This indicates that an instructor may advise at most one student, and a student may have at most one advisor.
- **One-to-many:** We draw a directed line from the relationship set *advisor* to the entity set *instructor* and an undirected line to the entity set *student* (see Figure 7.9b). This indicates that an instructor may advise many students, but a student may have at most one advisor.
- **Many-to-one:** We draw an undirected line from the relationship set *advisor* to the entity set *instructor* and a directed line to the entity set *student*. This indicates that an instructor may advise at most one student, but a student may have many advisors.
- **Many-to-many:** We draw an undirected line from the relationship set *advisor* to both entity sets *instructor* and *student* (see Figure 7.9c). This indicates that

Screen clipping taken: 13-06-2026 18:54

Weak Entity Sets:

An entity set that does not have sufficient attributes to form a primary key is termed a **weak entity set**.
 An entity set that has a primary key is termed a **strong entity set**.

For a weak entity set to be meaningful, it must be associated with another entity set, called the identifying or owner entity set.



Figure 7.14 E-R diagram with a weak entity set.

In E-R diagrams, a weak entity set is depicted via a rectangle, like a strong entity set, but there are two main differences:

- The discriminator of a weak entity is underlined with a dashed, rather than a solid, line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.

Screen clipping taken: 13-06-2026 19:05

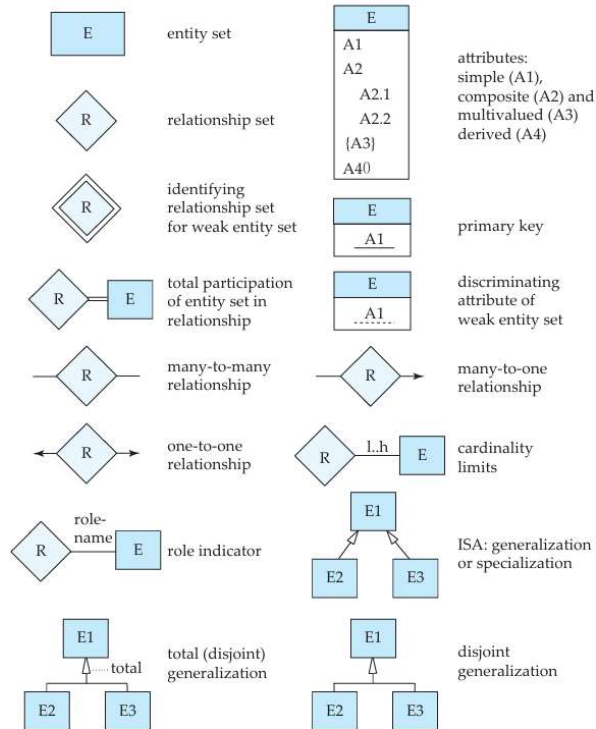


Figure 7.24 Symbols used in the E-R notation.

Screen clipping taken: 13-06-2026 19:13

Extended E-R Features:

extended E-R features of
specialization,
generalization,
higher- and lower-level entity sets,
attribute inheritance, and
aggregation.

The process of designating subgroupings within an entity set is called **specialization**. The specialization of person allows us to distinguish among person entities according to whether they correspond to employees or students: in general, a person could be an employee, a student, both, or neither.

This commonality can be expressed by **generalization**, which is a containment relationship that exists between a higher-level entity set and one or more lower-level entity sets.

Higher- and lower-level entity sets also may be designated by the terms **superclass and subclass**, respectively. The person entity set is the superclass of the employee and student subclasses.

A crucial property of the higher-and lower-level entities created by specialization and generalization is attribute inheritance. The attributes of the higher-level entity sets are said to be inherited by the lower-level entity sets.

Other Aspects of Database Design:

1. Data Constraints and Relational Database Design
2. Usage Requirements: Queries, Performance
3. Authorization Requirements
4. Data Flow, Workflow
5. Other Issues in Database Design

We have seen a variety of data constraints that can be expressed using SQL, including primary-key constraints, foreign-key constraints, check constraints, assertions, and triggers. Constraints serve several purposes. The most obvious one is the automation of consistency preservation. By expressing constraints in the SQL data-definition language, the designer is able to ensure that the database system itself enforces the constraints.

A further advantage of stating constraints explicitly is that certain constraints are particularly useful in designing relational database schemas

Database system performance is a critical aspect of most enterprise information systems

- Throughput—the number of queries or updates (often referred to as transactions) that can be processed on average per unit of time.
- Response time—the amount of time a single transaction takes from start to finish in either the average case or the worst case.

Authorization constraints affect design of the database as well because SQL allows access to be granted to users on the basis of components of the logical design of the database.